Experiment 2: Trust ~ Lines Model Comparison (0325)

Goal

Recreate Experiment 2 data loading and preprocessing from `0325.ipynb`, then compare:

- `Intercept-only (minimal): trust ~ 1`
- `Linear: trust ~ lines`
- `Quadratic: trust ~ poly(lines, 2, raw = TRUE)`
- `Cubic: trust ~ poly(lines, 3, raw = TRUE)`

Metrics:

- MSE
- LOOCV-MSE
- BIC
- AIC
- Lasso-regularized MSE

Packages

```
required_packages <- c(
  "dplyr",
  "jsonlite",
  "ggplot2",
  "knitr"
)

missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]

if (length(missing_packages) > 0) {
  stop(
    "Missing required packages: ",
    paste(missing_packages, collapse = ", "),
    ". Install them before rendering."
  )
}

lapply(required_packages, library, character.only = TRUE)
```

```
## [[1]]
## [1] "dplyr"     "stats"     "graphics"  "grDevices" "utils"     "datasets"
## [7] "methods"   "base"
##
## [[2]]
## [1] "jsonlite"  "dplyr"     "stats"     "graphics"  "grDevices" "utils"
## [7] "datasets"  "methods"   "base"
##
## [[3]]
##  [1] "ggplot2"   "jsonlite"  "dplyr"     "stats"     "graphics"  "grDevices"
##  [7] "utils"     "datasets"  "methods"   "base"
##
## [[4]]
##  [1] "knitr"     "ggplot2"   "jsonlite"  "dplyr"     "stats"     "graphics"
##  [7] "grDevices" "utils"     "datasets"  "methods"   "base"
```

Paths

```
report_dir <- local({
  current_input <- tryCatch(
    knitr::current_input(),
    error = function(e) character(0)
  )

  if (length(current_input) == 1 && nzchar(current_input)) {
    input_path <- current_input

    if (!grepl("^(/|[A-Za-z]:[/\\\\])", input_path)) {
      input_path <- file.path(getwd(), input_path)
    }

    if (grepl("\\.[Rr]md$", input_path) || file.exists(input_path)) {
      return(normalizePath(dirname(input_path), winslash = "/", mustWork = TRUE))
    }
  }

  fallback_dir <- file.path(getwd(), "Experiment2", "analysis")
  normalizePath(fallback_dir, winslash = "/", mustWork = TRUE)
})

resolve_report_path <- function(path_value) {
  if (grepl("^(/|[A-Za-z]:[/\\\\])", path_value)) {
    return(normalizePath(path_value, winslash = "/", mustWork = TRUE))
  }

  normalizePath(
    file.path(report_dir, path_value),
    winslash = "/",
    mustWork = TRUE
  )
}

data_dir_candidates <- vapply(params$data_dir_candidates, resolve_report_path, character(1))
existing_data_dirs <- data_dir_candidates[dir.exists(data_dir_candidates)]

if (length(existing_data_dirs) == 0) {
  stop(
    "No Experiment 2 data directory found. Checked: ",
    paste(data_dir_candidates, collapse = ", ")
  )
}

data_dir <- existing_data_dirs[[1]]
```

Constants

```
CONDITION_LABELS <- c(
  "1" = "Baseline",
  "2" = "CI",
  "3" = "2 Lines",
  "4" = "3 Lines",
  "5" = "4 Lines",
  "6" = "5 Lines",
  "7" = "6 Lines",
  "8" = "7 Lines",
  "9" = "8 Lines",
  "10" = "9 Lines"
)

BASELINE_CONDITION_NUMBER <- 1
TREATMENT_CONDITION_NUMBERS <- c(2, 3, 4, 5, 6, 7, 8, 9, 10)
TREATMENT_ORDER <- c("CI", "2 Lines", "3 Lines", "4 Lines", "5 Lines", "6 Lines", "7 Lines", "8 Lines", "9 Line
s")

TRUST_QUESTION_KEYS <- c(
  "skeptical_rating",
  "data_trust",
  "usability_difficulty",
  "comprehension_ease"
)

INTERACTION_QUESTION_KEYS <- c(
  "navigation_control",
  "content_control",
  "pace_control",
  "interface_exploration",
  "interface_responsiveness",
  "user_communication",
  "personal_conversation",
  "interface_interaction",
  "interface_sensitivity"
)

NORMALIZED_FLAG_COLUMN <- "_cousineau_normalized"
MOREY_FACTOR_COLUMN <- "_morey_factor"
K_CONDITIONS_COLUMN <- "_k_conditions"
LASSO_LAMBDA <- 0.05
```

Helper Functions

```
extract_condition_number <- function(condition_id) {
  if (is.na(condition_id) || !nzchar(as.character(condition_id))) {
    return(NA_real_)
  }

  raw <- sub(".*condition_([0-9]+).*", "\\1", as.character(condition_id), perl = TRUE)
  if (!grepl("^[0-9]+$", raw)) {
    return(NA_real_)
  }

  as.numeric(raw)
}

map_condition_labels <- function(condition_numbers) {
  vapply(
    condition_numbers,
    function(x) {
      key <- as.character(suppressWarnings(as.integer(x)))
      if (is.na(x) || !(key %in% names(CONDITION_LABELS))) {
        return("Other")
      }
      CONDITION_LABELS[[key]]
    },
    character(1)
  )
}

extract_participant_id_from_filename <- function(file_path) {
  parts <- strsplit(tools::file_path_sans_ext(basename(file_path)), "_", fixed = TRUE)[[1]]
  if (length(parts) >= 2) {
    return(parts[[2]])
  }
  NA_character_
}

parse_response_json <- function(value) {
  if (is.null(value) || length(value) == 0 || is.na(value) || !nzchar(as.character(value))) {
    return(list())
  }

  if (is.list(value) && !is.null(names(value))) {
    return(value)
  }

  parsed <- tryCatch(
    jsonlite::fromJSON(as.character(value), simplifyVector = TRUE),
    error = function(e) NULL
  )

  if (is.null(parsed) || !is.list(parsed)) {
    return(list())
  }

  parsed
}

safe_get_scalar <- function(x) {
  if (is.null(x) || length(x) == 0) {
    return(NA_character_)
  }

  as.character(x[[1]])
```

```
}

expand_response_columns <- function(data, response_col = "response") {
  if (!(response_col %in% names(data))) {
    return(data)
  }

  parsed_list <- lapply(data[[response_col]], parse_response_json)
  all_keys <- unique(unlist(lapply(parsed_list, names), use.names = FALSE))

  if (length(all_keys) == 0) {
    return(data)
  }

  expanded <- data.frame(
    lapply(all_keys, function(key) {
      vapply(parsed_list, function(item) safe_get_scalar(item[[key]]), character(1))
    }),
    stringsAsFactors = FALSE,
    check.names = FALSE
  )
  names(expanded) <- all_keys

  merged <- data
  for (col in names(expanded)) {
    parsed_values <- expanded[[col]]

    if (col %in% names(merged)) {
      existing <- merged[[col]]
      existing_chr <- trimws(as.character(existing))
      fill_mask <- is.na(existing) | existing_chr == ""
      existing[fill_mask] <- parsed_values[fill_mask]
      merged[[col]] <- existing
    } else {
      merged[[col]] <- parsed_values
    }
  }

  merged
}

identify_attention_check_failures <- function(
  data,
  trust_keys = TRUST_QUESTION_KEYS,
  interaction_keys = INTERACTION_QUESTION_KEYS,
  survey_trial_type = "trust-survey"
) {
  empty_result <- list(ids = character(0), details = data.frame())

  if (is.null(data) || nrow(data) == 0) {
    return(empty_result)
  }

  if (!all(c("trial_type", "participant_id") %in% names(data))) {
    return(empty_result)
  }

  survey_rows <- data[data$trial_type == survey_trial_type, , drop = FALSE]
  if (nrow(survey_rows) == 0) {
    return(empty_result)
  }

  survey_rows <- expand_response_columns(survey_rows)
  parsed_response <- if ("response" %in% names(survey_rows)) {
    lapply(survey_rows$response, parse_response_json)
```

```
  } else {
    replicate(nrow(survey_rows), list(), simplify = FALSE)
  }

  for (key in c(trust_keys, interaction_keys)) {
    existing <- if (key %in% names(survey_rows)) {
      suppressWarnings(as.numeric(survey_rows[[key]]))
    } else {
      rep(NA_real_, nrow(survey_rows))
    }

    if (sum(!is.na(existing)) == 0) {
      extracted <- vapply(parsed_response, function(item) safe_get_scalar(item[[key]]), character(1))
      existing <- suppressWarnings(as.numeric(extracted))
    }

    survey_rows[[key]] <- existing
  }

  trust_complete <- rowSums(is.na(survey_rows[, trust_keys, drop = FALSE])) == 0
  interaction_complete <- rowSums(is.na(survey_rows[, interaction_keys, drop = FALSE])) == 0

  trust_uniform <- apply(
    survey_rows[, trust_keys, drop = FALSE],
    1,
    function(row) {
      values <- unique(row[!is.na(row)])
      length(values) == 1
    }
  )

  interaction_uniform <- apply(
    survey_rows[, interaction_keys, drop = FALSE],
    1,
    function(row) {
      values <- unique(row[!is.na(row)])
      length(values) == 1
    }
  )

  failure_mask <- trust_complete & interaction_complete & trust_uniform & interaction_uniform
  failure_rows <- survey_rows[failure_mask & !is.na(survey_rows$participant_id), , drop = FALSE]

  if (nrow(failure_rows) == 0) {
    return(empty_result)
  }

  failure_rows$participant_id <- as.character(failure_rows$participant_id)
  detail_columns <- c(
    "participant_id",
    "condition_id",
    "condition_number",
    trust_keys,
    interaction_keys
  )
  available_detail_columns <- detail_columns[detail_columns %in% names(failure_rows)]

  list(
    ids = unique(failure_rows$participant_id),
    details = failure_rows[, available_detail_columns, drop = FALSE]
  )
}

normalize_scores_cousineau_morey <- function(
  data,
```

```
  metric_columns,
  baseline_condition = BASELINE_CONDITION_NUMBER,
  drop_baseline = TRUE,
  normalization_conditions = NULL,
  require_complete_participants = TRUE
) {
  if (is.null(data) || nrow(data) == 0) {
    normalized <- if (is.null(data)) data.frame() else data
    if (nrow(normalized) > 0) {
      normalized[[NORMALIZED_FLAG_COLUMN]] <- FALSE
      normalized[[MOREY_FACTOR_COLUMN]] <- NA_real_
      normalized[[K_CONDITIONS_COLUMN]] <- NA_real_
    }
    return(normalized)
  }

  normalized <- data

  if (!all(c("participant_id", "condition_number") %in% names(normalized))) {
    normalized[[NORMALIZED_FLAG_COLUMN]] <- FALSE
    normalized[[MOREY_FACTOR_COLUMN]] <- NA_real_
    normalized[[K_CONDITIONS_COLUMN]] <- NA_real_
    return(normalized)
  }

  available_columns <- metric_columns[metric_columns %in% names(normalized)]
  normalized$condition_number <- suppressWarnings(as.numeric(normalized$condition_number))
  normalized$.row_id <- seq_len(nrow(normalized))
  normalized[[NORMALIZED_FLAG_COLUMN]] <- FALSE
  normalized[[MOREY_FACTOR_COLUMN]] <- NA_real_
  normalized[[K_CONDITIONS_COLUMN]] <- NA_real_

  return_without_norm <- function(df) {
    df$.row_id <- NULL
    if (drop_baseline) {
      df <- df[df$condition_number != baseline_condition, , drop = FALSE]
    }
    df
  }

  if (length(available_columns) == 0) {
    return(return_without_norm(normalized))
  }

  if (is.null(normalization_conditions)) {
    observed_conditions <- sort(unique(stats::na.omit(as.integer(normalized$condition_number))))
    normalization_conditions <- observed_conditions[observed_conditions != as.integer(baseline_condition)]
  } else {
    normalization_conditions <- sort(unique(as.integer(normalization_conditions)))
  }

  if (length(normalization_conditions) == 0) {
    return(return_without_norm(normalized))
  }

  normalization_data <- normalized[normalized$condition_number %in% normalization_conditions, , drop = FALSE]
  if (nrow(normalization_data) == 0) {
    return(return_without_norm(normalized))
  }

  expected_k <- length(normalization_conditions)

  if (require_complete_participants) {
    participant_condition_counts <- stats::aggregate(
      condition_number ~ participant_id,
```

```
      data = normalization_data,
      FUN = function(x) length(unique(x[!is.na(x)]))
    )

    complete_participants <- as.character(
      participant_condition_counts$participant_id[
        participant_condition_counts$condition_number >= expected_k
      ]
    )

    normalization_data <- normalization_data[
      as.character(normalization_data$participant_id) %in% complete_participants,
      ,
      drop = FALSE
    ]
  }

  if (nrow(normalization_data) == 0) {
    return(return_without_norm(normalized))
  }

  normalization_row_ids <- normalization_data$.row_id

  for (col in available_columns) {
    numeric_values <- suppressWarnings(as.numeric(normalization_data[[col]]))
    participant_means <- ave(
      numeric_values,
      normalization_data$participant_id,
      FUN = function(x) {
        mean_value <- mean(x, na.rm = TRUE)
        if (is.nan(mean_value)) {
          return(NA_real_)
        }
        mean_value
      }
    )

    grand_mean <- mean(numeric_values, na.rm = TRUE)
    if (is.nan(grand_mean)) {
      grand_mean <- NA_real_
    }

    normalized_values <- numeric_values - participant_means + grand_mean
    normalized[[col]][normalization_row_ids] <- normalized_values
  }

  morey_factor <- if (expected_k > 1) sqrt(expected_k / (expected_k - 1)) else NA_real_
  normalized[[NORMALIZED_FLAG_COLUMN]][normalization_row_ids] <- TRUE
  normalized[[MOREY_FACTOR_COLUMN]][normalization_row_ids] <- morey_factor
  normalized[[K_CONDITIONS_COLUMN]][normalization_row_ids] <- expected_k

  return_without_norm(normalized)
}

ensure_likert_columns <- function(data, keys) {
  if (is.null(data) || nrow(data) == 0) {
    return(data)
  }

  working <- data
  parsed <- if ("response" %in% names(working)) {
    lapply(working$response, parse_response_json)
  } else {
    replicate(nrow(working), list(), simplify = FALSE)
  }
```

```
  for (key in keys) {
    existing <- if (key %in% names(working)) {
      suppressWarnings(as.numeric(working[[key]]))
    } else {
      rep(NA_real_, nrow(working))
    }

    if (!(key %in% names(working)) || sum(!is.na(existing)) == 0) {
      extracted <- suppressWarnings(as.numeric(
        vapply(parsed, function(item) safe_get_scalar(item[[key]]), character(1))
      ))

      valid <- extracted[!is.na(extracted)]
      if (length(valid) > 0 && all(valid >= 0 & valid <= 6)) {
        extracted <- extracted + 1
      }

      working[[key]] <- extracted
    } else {
      working[[key]] <- existing
    }
  }

  working
}

compute_composite_scores <- function(data) {
  if (is.null(data) || nrow(data) == 0) {
    return(data.frame())
  }

  working <- data
  for (col in c(TRUST_QUESTION_KEYS, INTERACTION_QUESTION_KEYS)) {
    if (col %in% names(working)) {
      working[[col]] <- suppressWarnings(as.numeric(working[[col]]))
    }
  }

  if (all(c(
    "skeptical_rating",
    "data_trust",
    "usability_difficulty",
    "comprehension_ease"
  ) %in% names(working))) {
    working$skeptical_rating_rev <- 8 - working$skeptical_rating
    working$usability_difficulty_rev <- 8 - working$usability_difficulty
    working$composite_trust_score <- rowMeans(
      working[, c(
        "skeptical_rating_rev",
        "data_trust",
        "usability_difficulty_rev",
        "comprehension_ease"
      ), drop = FALSE],
      na.rm = TRUE
    )
    working$composite_trust_score[is.nan(working$composite_trust_score)] <- NA_real_
  }

  interaction_available <- INTERACTION_QUESTION_KEYS[
    INTERACTION_QUESTION_KEYS %in% names(working)
  ]

  if (length(interaction_available) > 0) {
    working$composite_interaction_score <- rowMeans(
```

```
      working[, interaction_available, drop = FALSE],
      na.rm = TRUE
    )
    working$composite_interaction_score[is.nan(working$composite_interaction_score)] <- NA_real_
  }

  working
}

spaced_kable <- function(x, ...) {
  knitr::kable(
    x,
    format = "html",
    table.attr = 'style="border-collapse: separate; border-spacing: 18px 6px;"',
    ...
  )
}
```

Data Loading and Preprocessing (Matched to 0325.ipynb)

```
csv_files <- sort(list.files(
  data_dir,
  pattern = params$csv_pattern,
  full.names = TRUE
))

if (length(csv_files) == 0) {
  stop(
    "No CSV files found in ",
    data_dir,
    " with pattern ",
    params$csv_pattern
  )
}

all_data <- lapply(csv_files, function(file_path) {
  df <- read.csv(file_path, stringsAsFactors = FALSE)
  df$source_file <- basename(file_path)

  if (!("participant_id" %in% names(df)) || all(is.na(df$participant_id))) {
    df$participant_id <- extract_participant_id_from_filename(file_path)
  }

  if (!("condition_id" %in% names(df))) {
    df$condition_id <- NA_character_
  }

  df$condition_number <- vapply(df$condition_id, extract_condition_number, numeric(1))
  df$condition_label <- map_condition_labels(df$condition_number)
  df
})

combined_data <- bind_rows(all_data)

before_participants <- dplyr::n_distinct(combined_data$participant_id, na.rm = TRUE)
combined_data <- combined_data[!is.na(combined_data$participant_id), , drop = FALSE]
combined_data <- combined_data[
  tolower(as.character(combined_data$participant_id)) != "test",
  ,
  drop = FALSE
]

attention_failures <- identify_attention_check_failures(combined_data)
if (length(attention_failures$ids) > 0) {
  combined_data <- combined_data[
    !(as.character(combined_data$participant_id) %in% attention_failures$ids),
    ,
    drop = FALSE
  ]
}

after_participants <- dplyr::n_distinct(combined_data$participant_id, na.rm = TRUE)

cat("Data directory:", data_dir, "\n")
```

```
## Data directory: /Users/songwen/Documents/Code Lab/Interact4Trust/Experiment2/data/0325/data
```

```
cat("Found CSV files:", length(csv_files), "\n")
```

```
## Found CSV files: 91
```

```
cat("Combined shape:", paste(dim(combined_data), collapse = " x "), "\n")
```

```
## Combined shape: 4872 x 74
```

```
cat("Participants:", before_participants, "->", after_participants, "\n")
```

```
## Participants: 91 -> 87
```

```
cat("Attention-check exclusions:", length(attention_failures$ids), "\n\n")
```

```
## Attention-check exclusions: 4
```

```
condition_lookup_columns <- c("condition_number", "condition_id", "condition_label", "condition_name")
condition_lookup_columns <- condition_lookup_columns[condition_lookup_columns %in% names(combined_data)]

condition_lookup <- combined_data %>%
  dplyr::select(dplyr::all_of(condition_lookup_columns)) %>%
  dplyr::filter(!is.na(condition_id)) %>%
  dplyr::distinct() %>%
  dplyr::arrange(condition_number, condition_id)

spaced_kable(condition_lookup)
```

condition_number	condition_id	condition_label	condition_name
1	condition_1_baseline_aggregation	Baseline	baseline_aggregation_only_odd_hover_show_one
1	condition_1_baseline_aggregation	Baseline	baseline_aggregation_only_even_click_show_one
1	condition_1_baseline_aggregation	Baseline	baseline_aggregation_only_odd_click_show_one
1	condition_1_baseline_aggregation	Baseline	baseline_aggregation_only_even_hover_show_one
2	condition_2_ci_95	CI	ci_95_both_cities_odd_hover_show_one
2	condition_2_ci_95	CI	ci_95_both_cities_even_click_show_one
2	condition_2_ci_95	CI	ci_95_both_cities_odd_click_show_one
2	condition_2_ci_95	CI	ci_95_both_cities_even_hover_show_one
3	condition_3_ensemble_2_lines	2 Lines	ensemble_2_lines_per_city_odd_hover_show_one
3	condition_3_ensemble_2_lines	2 Lines	ensemble_2_lines_per_city_even_click_show_one
3	condition_3_ensemble_2_lines	2 Lines	ensemble_2_lines_per_city_odd_click_show_one
3	condition_3_ensemble_2_lines	2 Lines	ensemble_2_lines_per_city_even_hover_show_one
4	condition_4_ensemble_3_lines	3 Lines	ensemble_3_lines_per_city_odd_hover_show_one
4	condition_4_ensemble_3_lines	3 Lines	ensemble_3_lines_per_city_even_click_show_one
4	condition_4_ensemble_3_lines	3 Lines	ensemble_3_lines_per_city_odd_click_show_one
4	condition_4_ensemble_3_lines	3 Lines	ensemble_3_lines_per_city_even_hover_show_one
5	condition_5_ensemble_4_lines	4 Lines	ensemble_4_lines_per_city_odd_hover_show_one
5	condition_5_ensemble_4_lines	4 Lines	ensemble_4_lines_per_city_even_click_show_one
5	condition_5_ensemble_4_lines	4 Lines	ensemble_4_lines_per_city_odd_click_show_one
5	condition_5_ensemble_4_lines	4 Lines	ensemble_4_lines_per_city_even_hover_show_one
6	condition_6_ensemble_5_lines	5 Lines	ensemble_5_lines_per_city_odd_hover_show_one
6	condition_6_ensemble_5_lines	5 Lines	ensemble_5_lines_per_city_even_click_show_one
6	condition_6_ensemble_5_lines	5 Lines	ensemble_5_lines_per_city_odd_click_show_one
6	condition_6_ensemble_5_lines	5 Lines	ensemble_5_lines_per_city_even_hover_show_one
7	condition_7_ensemble_6_lines	6 Lines	ensemble_6_lines_per_city_odd_hover_show_one
7	condition_7_ensemble_6_lines	6 Lines	ensemble_6_lines_per_city_even_click_show_one

condition_number	condition_id	condition_label	condition_name
7	condition_7_ensemble_6_lines	6 Lines	ensemble_6_lines_per_city_odd_click_show_one
7	condition_7_ensemble_6_lines	6 Lines	ensemble_6_lines_per_city_even_hover_show_one
8	condition_8_ensemble_7_lines	7 Lines	ensemble_7_lines_per_city_odd_hover_show_one
8	condition_8_ensemble_7_lines	7 Lines	ensemble_7_lines_per_city_even_click_show_one
8	condition_8_ensemble_7_lines	7 Lines	ensemble_7_lines_per_city_odd_click_show_one
8	condition_8_ensemble_7_lines	7 Lines	ensemble_7_lines_per_city_even_hover_show_one
9	condition_9_ensemble_8_lines	8 Lines	ensemble_8_lines_per_city_odd_hover_show_one
9	condition_9_ensemble_8_lines	8 Lines	ensemble_8_lines_per_city_even_click_show_one
9	condition_9_ensemble_8_lines	8 Lines	ensemble_8_lines_per_city_odd_click_show_one
9	condition_9_ensemble_8_lines	8 Lines	ensemble_8_lines_per_city_even_hover_show_one
10	condition_10_ensemble_9_lines	9 Lines	ensemble_9_lines_per_city_odd_hover_show_one
10	condition_10_ensemble_9_lines	9 Lines	ensemble_9_lines_per_city_even_click_show_one
10	condition_10_ensemble_9_lines	9 Lines	ensemble_9_lines_per_city_odd_click_show_one
10	condition_10_ensemble_9_lines	9 Lines	ensemble_9_lines_per_city_even_hover_show_one
NA		Other	

```
trust_survey_rows <- combined_data %>%
  dplyr::filter(trial_type == "trust-survey")

trust_survey_expanded <- ensure_likert_columns(
  trust_survey_rows,
  c(TRUST_QUESTION_KEYS, INTERACTION_QUESTION_KEYS)
)

analysis_metric_columns <- c(TRUST_QUESTION_KEYS, INTERACTION_QUESTION_KEYS)
analysis_metric_columns <- analysis_metric_columns[
  analysis_metric_columns %in% names(trust_survey_expanded)
]

analysis_trust_survey_expanded <- normalize_scores_cousineau_morey(
  trust_survey_expanded,
  metric_columns = analysis_metric_columns,
  baseline_condition = BASELINE_CONDITION_NUMBER,
  drop_baseline = FALSE,
  normalization_conditions = TREATMENT_CONDITION_NUMBERS,
  require_complete_participants = TRUE
)

analysis_treatment_survey <- analysis_trust_survey_expanded %>%
  dplyr::filter(
    condition_number %in% TREATMENT_CONDITION_NUMBERS,
    !is.na(.data[[NORMALIZED_FLAG_COLUMN]]) & .data[[NORMALIZED_FLAG_COLUMN]]
  )

cat("Trust-survey rows:", nrow(trust_survey_rows), "\n")
```

```
## Trust-survey rows: 870
```

```
cat("Rows with parsed Likert columns:", nrow(trust_survey_expanded), "\n")
```

```
## Rows with parsed Likert columns: 870
```

```
cat("Normalized treatment rows kept:", nrow(analysis_treatment_survey), "\n")
```

```
## Normalized treatment rows kept: 783
```

```
cat(
  "Participants retained after complete-condition requirement:",
  dplyr::n_distinct(analysis_treatment_survey$participant_id),
  "\n"
)
```

```
## Participants retained after complete-condition requirement: 87
```

```
composite_treatment <- compute_composite_scores(analysis_treatment_survey)
composite_treatment$condition_label <- map_condition_labels(composite_treatment$condition_number)

condition_level_summary <- composite_treatment %>%
  dplyr::filter(
    condition_number %in% TREATMENT_CONDITION_NUMBERS,
    !is.na(composite_trust_score),
    !is.na(composite_interaction_score)
  ) %>%
  dplyr::group_by(condition_number, condition_label) %>%
  dplyr::summarise(
    trust_mean = mean(composite_trust_score, na.rm = TRUE),
    trust_sd = stats::sd(composite_trust_score, na.rm = TRUE),
    interaction_mean = mean(composite_interaction_score, na.rm = TRUE),
    interaction_sd = stats::sd(composite_interaction_score, na.rm = TRUE),
    n = dplyr::n(),
    .groups = "drop"
  )

available_morey <- if (MOREY_FACTOR_COLUMN %in% names(composite_treatment)) {
  suppressWarnings(as.numeric(composite_treatment[[MOREY_FACTOR_COLUMN]]))
} else {
  numeric(0)
}
available_morey <- available_morey[!is.na(available_morey)]
morey_factor <- if (length(available_morey) > 0) available_morey[[1]] else NA_real_

if (!is.na(morey_factor)) {
  condition_level_summary$trust_sd <- condition_level_summary$trust_sd * morey_factor
  condition_level_summary$interaction_sd <- condition_level_summary$interaction_sd * morey_factor
}

condition_level_summary$trust_se <- condition_level_summary$trust_sd / sqrt(pmax(condition_level_summary$n, 1))
condition_level_summary$interaction_se <- condition_level_summary$interaction_sd / sqrt(pmax(condition_level_su
mmary$n, 1))

condition_level_summary$condition_label <- factor(
  condition_level_summary$condition_label,
  levels = TREATMENT_ORDER,
  ordered = TRUE
)
condition_level_summary <- condition_level_summary %>%
  dplyr::arrange(condition_label)

spaced_kable(
  condition_level_summary %>%
    dplyr::mutate(dplyr::across(where(is.numeric), ~ round(.x, 4)))
)
```

condition_number	condition_label	trust_mean	trust_sd	interaction_mean	interaction_sd	n	trust_se	interaction_se
2	CI	5.2414	0.9125	3.7944	0.5604	87	0.0978	0.0601

condition_number	condition_label	trust_mean	trust_sd	interaction_mean	interaction_sd	n	trust_se	interaction_se
3	2 Lines	5.1092	0.7825	3.8391	0.3781	87	0.0839	0.0405
4	3 Lines	5.1580	0.6527	3.8174	0.3896	87	0.0700	0.0418
5	4 Lines	4.9483	0.6932	3.7791	0.3755	87	0.0743	0.0403
6	5 Lines	4.9080	0.6827	3.7778	0.3971	87	0.0732	0.0426
7	6 Lines	5.2069	0.5646	3.8723	0.3425	87	0.0605	0.0367
8	7 Lines	5.0920	0.6782	3.8365	0.3772	87	0.0727	0.0404
9	8 Lines	4.9799	0.6787	3.7701	0.2650	87	0.0728	0.0284
10	9 Lines	5.0920	0.7331	3.7573	0.4683	87	0.0786	0.0502

```
cat(
  if (!is.na(morey_factor)) {
    paste0("Morey factor: ", round(morey_factor, 4))
  } else {
    "Morey factor: NA"
  },
  "\n"
)
```

```
## Morey factor: 1.0607
```

Regression Dataset (`trust ~ lines`)

```
regression_data <- condition_level_summary %>%
  dplyr::transmute(
    condition_number = as.numeric(condition_number),
    condition_label = as.character(condition_label),
    lines = condition_number - 1,
    trust = trust_mean
  ) %>%
  dplyr::arrange(lines)

spaced_kable(
  regression_data %>%
    dplyr::mutate(dplyr::across(where(is.numeric), ~ round(.x, 4)))
)
```

condition_number	condition_label	lines	trust
2	CI	1	5.2414
3	2 Lines	2	5.1092
4	3 Lines	3	5.1580
5	4 Lines	4	4.9483
6	5 Lines	5	4.9080
7	6 Lines	6	5.2069
8	7 Lines	7	5.0920
9	8 Lines	8	4.9799
10	9 Lines	9	5.0920

Fit Models and Compare (MSE, LOOCV-MSE, BIC, AIC, Lasso-

Regularized MSE)

```
model_specs <- list(
  "Intercept-only (minimal)" = function(df) lm(trust ~ 1, data = df),
  "Linear" = function(df) lm(trust ~ lines, data = df),
  "Quadratic" = function(df) lm(trust ~ poly(lines, 2, raw = TRUE), data = df),
  "Cubic" = function(df) lm(trust ~ poly(lines, 3, raw = TRUE), data = df)
)

loocv_mse <- function(data, fit_fn) {
  n <- nrow(data)
  squared_errors <- rep(NA_real_, n)

  for (i in seq_len(n)) {
    train <- data[-i, , drop = FALSE]
    test <- data[i, , drop = FALSE]

    fit <- tryCatch(fit_fn(train), error = function(e) NULL)
    if (is.null(fit)) {
      next
    }

    prediction <- tryCatch(
      as.numeric(predict(fit, newdata = test)),
      error = function(e) NA_real_
    )

    squared_errors[i] <- (test$trust - prediction)^2
  }

  mean(squared_errors, na.rm = TRUE)
}

calc_approx_aic <- function(y, y_hat, k) {
  n <- length(y)
  rss <- sum((y - y_hat)^2)
  rss <- max(rss, .Machine$double.eps)
  n * log(rss / n) + n * (1 + log(2 * pi)) + 2 * k
}

calc_approx_bic <- function(y, y_hat, k) {
  n <- length(y)
  rss <- sum((y - y_hat)^2)
  rss <- max(rss, .Machine$double.eps)
  n * log(rss / n) + n * (1 + log(2 * pi)) + log(n) * k
}

get_bic_value <- function(model, y, y_hat) {
  bic_value <- tryCatch(as.numeric(BIC(model)), error = function(e) NA_real_)
  if (is.finite(bic_value)) {
    return(bic_value)
  }

  effective_k <- length(stats::coef(model)) + 1
  calc_approx_bic(y, y_hat, effective_k)
}

get_aic_value <- function(model, y, y_hat) {
  aic_value <- tryCatch(as.numeric(AIC(model)), error = function(e) NA_real_)
  if (is.finite(aic_value)) {
    return(aic_value)
  }

  effective_k <- length(stats::coef(model)) + 1
```

```
  calc_approx_aic(y, y_hat, effective_k)
}

calc_lasso_regularized_mse <- function(model, mse, lambda = LASSO_LAMBDA) {
  coefs <- tryCatch(stats::coef(model), error = function(e) numeric(0))
  if (length(coefs) == 0) {
    return(NA_real_)
  }

  coef_names <- names(coefs)
  if (is.null(coef_names)) {
    penalty <- sum(abs(coefs), na.rm = TRUE)
  } else {
    penalty <- sum(abs(coefs[coef_names != "(Intercept)"]), na.rm = TRUE)
  }

  mse + lambda * penalty
}

evaluate_model <- function(model_name, fit_fn, data) {
  fit <- fit_fn(data)
  fitted_values <- as.numeric(predict(fit, newdata = data))
  mse <- mean((data$trust - fitted_values)^2, na.rm = TRUE)
  loocv <- loocv_mse(data, fit_fn)
  bic_value <- get_bic_value(fit, data$trust, fitted_values)
  aic_value <- get_aic_value(fit, data$trust, fitted_values)
  lasso_reg_mse <- calc_lasso_regularized_mse(fit, mse)

  list(
    fit = fit,
    metrics = data.frame(
      model = model_name,
      mse = mse,
      loocv_mse = loocv,
      bic = bic_value,
      aic = aic_value,
      lasso_regularized_mse = lasso_reg_mse,
      stringsAsFactors = FALSE
    )
  )
}

evaluated_models <- lapply(
  names(model_specs),
  function(model_name) evaluate_model(model_name, model_specs[[model_name]], regression_data)
)
names(evaluated_models) <- names(model_specs)

model_comparison <- dplyr::bind_rows(
  lapply(evaluated_models, function(x) x$metrics)
) %>%
  dplyr::arrange(loocv_mse)

best_model_name <- model_comparison$model[[1]]

model_comparison_pretty <- model_comparison %>%
  dplyr::mutate(
    mse = round(mse, 6),
    loocv_mse = round(loocv_mse, 6),
    bic = round(bic, 6),
    aic = round(aic, 6),
    lasso_regularized_mse = round(lasso_regularized_mse, 6)
  )

spaced_kable(
```

```
  model_comparison_pretty,
  col.names = c("Model", "MSE", "LOOCV-MSE", "BIC", "AIC", "Lasso-Regularized MSE")
)
```

Model	MSE	LOOCV-MSE	BIC	AIC	Lasso-Regularized MSE
Intercept-only (minimal)	0.011810	0.014947	-10.01388	-10.408330	0.011810
Quadratic	0.008389	0.015182	-8.69747	-9.486369	0.013366
Linear	0.010443	0.015964	-8.92384	-9.515514	0.011159
Cubic	0.007598	0.022216	-7.39183	-8.377953	0.021425

```
cat("Best model by LOOCV-MSE:", best_model_name, "\n")
```

```
## Best model by LOOCV-MSE: Intercept-only (minimal)
```

Regression Dataset (`interaction experience ~ lines`)

```
interaction_regression_data <- condition_level_summary %>%
  dplyr::transmute(
    condition_number = as.numeric(condition_number),
    condition_label = as.character(condition_label),
    lines = condition_number - 1,
    interaction = interaction_mean
  ) %>%
  dplyr::arrange(lines)

spaced_kable(
  interaction_regression_data %>%
    dplyr::mutate(dplyr::across(where(is.numeric), ~ round(.x, 4)))
)
```

condition_number	condition_label	lines	interaction
2	CI	1	3.7944
3	2 Lines	2	3.8391
4	3 Lines	3	3.8174
5	4 Lines	4	3.7791
6	5 Lines	5	3.7778
7	6 Lines	6	3.8723
8	7 Lines	7	3.8365
9	8 Lines	8	3.7701
10	9 Lines	9	3.7573

Fit Interaction Models and Compare (MSE, LOOCV-MSE, BIC,

AIC, Lasso-Regularized MSE)

```
interaction_model_specs <- list(
  "Intercept-only (minimal)" = function(df) lm(interaction ~ 1, data = df),
  "Linear" = function(df) lm(interaction ~ lines, data = df),
  "Quadratic" = function(df) lm(interaction ~ poly(lines, 2, raw = TRUE), data = df),
  "Cubic" = function(df) lm(interaction ~ poly(lines, 3, raw = TRUE), data = df)
)

loocv_mse_interaction <- function(data, fit_fn) {
  n <- nrow(data)
  squared_errors <- rep(NA_real_, n)

  for (i in seq_len(n)) {
    train <- data[-i, , drop = FALSE]
    test <- data[i, , drop = FALSE]

    fit <- tryCatch(fit_fn(train), error = function(e) NULL)
    if (is.null(fit)) {
      next
    }

    prediction <- tryCatch(
      as.numeric(predict(fit, newdata = test)),
      error = function(e) NA_real_
    )

    squared_errors[i] <- (test$interaction - prediction)^2
  }

  mean(squared_errors, na.rm = TRUE)
}

evaluate_interaction_model <- function(model_name, fit_fn, data) {
  fit <- fit_fn(data)
  fitted_values <- as.numeric(predict(fit, newdata = data))
  mse <- mean((data$interaction - fitted_values)^2, na.rm = TRUE)
  loocv <- loocv_mse_interaction(data, fit_fn)
  bic_value <- get_bic_value(fit, data$interaction, fitted_values)
  aic_value <- get_aic_value(fit, data$interaction, fitted_values)
  lasso_reg_mse <- calc_lasso_regularized_mse(fit, mse)

  list(
    fit = fit,
    metrics = data.frame(
      model = model_name,
      mse = mse,
      loocv_mse = loocv,
      bic = bic_value,
      aic = aic_value,
      lasso_regularized_mse = lasso_reg_mse,
      stringsAsFactors = FALSE
    )
  )
}

interaction_evaluated_models <- lapply(
  names(interaction_model_specs),
  function(model_name) evaluate_interaction_model(
    model_name,
    interaction_model_specs[[model_name]],
    interaction_regression_data
  )
)
```

```r
names(interaction_evaluated_models) <- names(interaction_model_specs)

interaction_model_comparison <- dplyr::bind_rows(
  lapply(interaction_evaluated_models, function(x) x$metrics)
) %>%
  dplyr::arrange(loocv_mse)

interaction_best_model_name <- interaction_model_comparison$model[[1]]

interaction_model_comparison_pretty <- interaction_model_comparison %>%
  dplyr::mutate(
    mse = round(mse, 6),
    loocv_mse = round(loocv_mse, 6),
    bic = round(bic, 6),
    aic = round(aic, 6),
    lasso_regularized_mse = round(lasso_regularized_mse, 6)
  )

spaced_kable(
  interaction_model_comparison_pretty,
  col.names = c("Model", "MSE", "LOOCV-MSE", "BIC", "AIC", "Lasso-Regularized MSE")
)
```

Model	MSE	LOOCV-MSE	BIC	AIC	Lasso-Regularized MSE
Intercept-only (minimal)	0.001317	0.001667	-29.75797	-30.15242	0.001317
Quadratic	0.001039	0.001901	-27.49136	-28.28026	0.002131
Linear	0.001224	0.001937	-28.21631	-28.80799	0.001410
Cubic	0.000898	0.004062	-26.61371	-27.59983	0.003552

```r
cat("Best interaction model by LOOCV-MSE:", interaction_best_model_name, "\n")
```

```
## Best interaction model by LOOCV-MSE: Intercept-only (minimal)
```

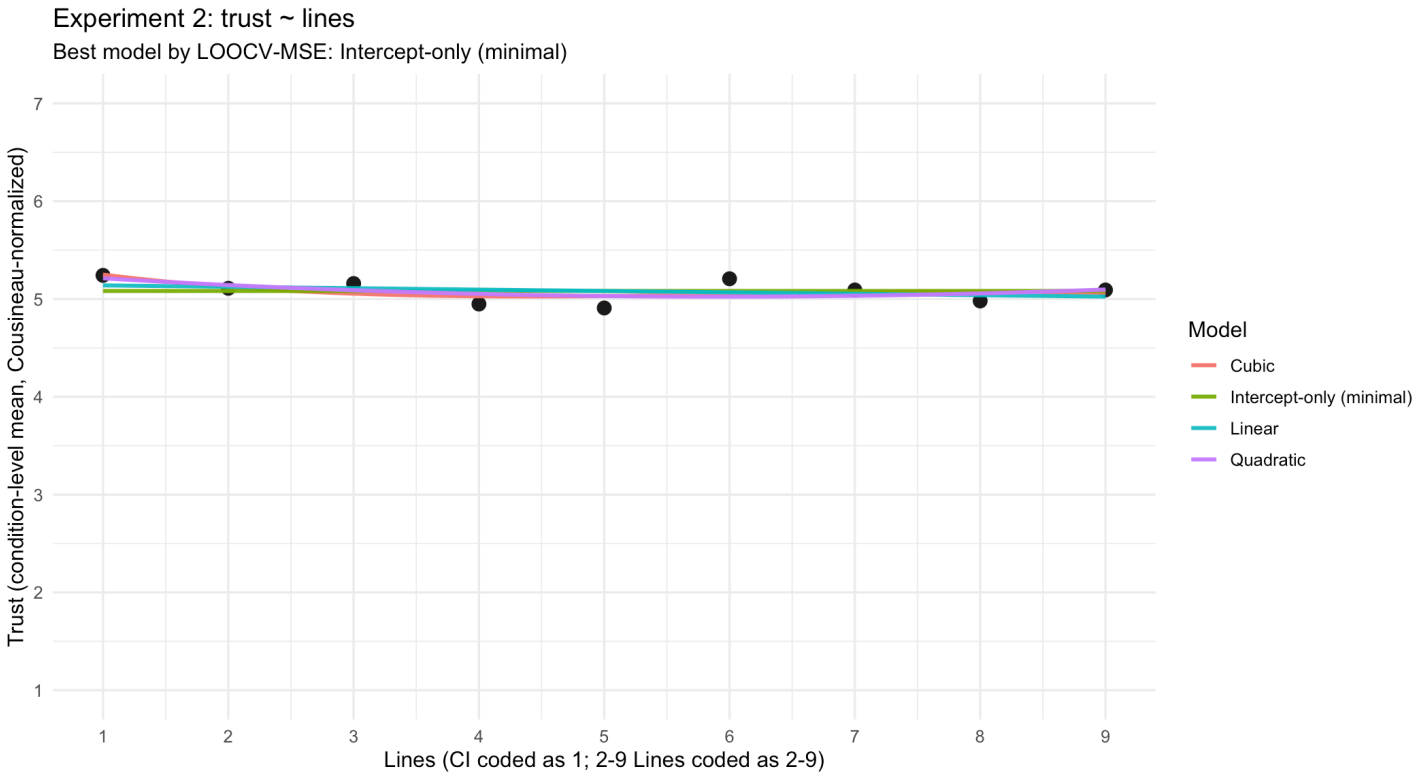
Plot Fitted Curves Over Observed Data

```
prediction_grid <- data.frame(
  lines = seq(min(regression_data$lines), max(regression_data$lines), length.out = 250)
)

fitted_curves <- dplyr::bind_rows(
  lapply(names(evaluated_models), function(model_name) {
    fit <- evaluated_models[[model_name]]$fit
    preds <- suppressWarnings(as.numeric(predict(fit, newdata = prediction_grid)))

    data.frame(
      model = model_name,
      lines = prediction_grid$lines,
      trust = preds,
      stringsAsFactors = FALSE
    )
  })
)

ggplot(regression_data, aes(x = lines, y = trust)) +
  geom_point(size = 3.2, color = "#1f1f1f") +
  geom_line(
    data = fitted_curves,
    aes(x = lines, y = trust, color = model),
    linewidth = 1.05,
    alpha = 0.95
  ) +
  scale_x_continuous(breaks = regression_data$lines) +
  scale_y_continuous(breaks = seq(1, 7, by = 1)) +
  coord_cartesian(ylim = c(1, 7)) +
  labs(
    title = "Experiment 2: trust ~ lines",
    subtitle = paste("Best model by LOOCV-MSE:", best_model_name),
    x = "Lines (CI coded as 1; 2-9 Lines coded as 2-9)",
    y = "Trust (condition-level mean, Cousineau-normalized)",
    color = "Model"
  ) +
  theme_minimal(base_size = 12)
```


Experiment 2: trust ~ lines
Best model by LOOCV-MSE: Intercept-only (minimal)
Trust (condition-level mean, Cousineau-normalized)
7
6
5
4
3
2
1
1
2
3
4
5
6
7
8
9
Lines (CI coded as 1; 2-9 Lines coded as 2-9)
Model
Cubic
Intercept-only (minimal)
Linear
Quadratic